`head`, then does a compare/exchange loop on `head` to increase the reference count, and then reads the `node` structure to obtain the `next` value. Right here you can see a required relationship; the `next` value is a plain nonatomic object, so in order to read this safely, there must be a happens-before relationship between the store (by the pushing thread) and the load (by the popping thread). Because the only atomic operation in the `push()` is the `compare_exchange_weak()`, and you need a *release* operation to get a happens-before relationship between threads, the `compare_exchange_weak()` must be `std::memory_order_release` or stronger. If the `compare_exchange_weak()` call fails, nothing has changed and you keep looping, so you need only `std::memory_order_relaxed` in that case:

```
void push(T const& data)
{
    counted_node_ptr new_node;
    new_node.ptr=new node(data);
    new_node.external_count=1;
    new_node.ptr->next=head.load(std::memory_order_relaxed)
    while(!head.compare_exchange_weak(new_node.ptr->next,new_node,
        std::memory_order_release,std::memory_order_relaxed));
}
```

What about the `pop()` code? In order to get the happens-before relationship you need, you must have an operation that's `std::memory_order_acquire` or stronger before the access to `next`. The pointer you dereference to access the `next` field is the old value read by the `compare_exchange_strong()` in `increase_head_count()`, so you need the ordering on that if it succeeds. As with the call in `push()`, if the exchange fails, you just loop again, so you can use relaxed ordering on failure:

```
void increase_head_count(counted_node_ptr& old_counter)
{
    counted_node_ptr new_counter;

    do
    {
        new_counter=old_counter;
        ++new_counter.external_count;
```